

Assignment Kit for Size Counting Standard



Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

This material approved for public release.
Distribution is limited by the Software Engineering Institute to attendees.

Personal Software Process for Engineers: Part I

Assignment Kit for the Size Counting Standard

Overview

Overview This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Size counting standard requirements	3
Example 1: Pascal size counting standard	4
Example 2: C++ size counting standard	6
Example 3: another size counting standard	8
Evaluation criteria and suggestions	10
Size counting standard template	11

Prerequisites Prerequisite reading

- Chapter 3

Objectives The objectives of Size Counting standard are to

- define the size counting standards that are appropriate for the programming language and environment that you will use
- provide a basis for developing a coding standard
- prepare for developing a program to count program size

Size counting standard requirements

Size counting standard requirements

Produce and document a standard for counting program size for the language and environment that you will use in this course.

Submit the completed standard using the format in the template on page 11 with your program 2 assignment package.

Example 1: Pascal size counting standard

Overview

The template is a simplified version of the SEI measurement framework.

- Use this template to describe important items.
- Tailor it to fit your needs or language.

We'll walk through two example size counting standards. The first example is for logical LOC for Pascal programs.

Completing the header

Complete the header as follows:

- the name you give this standard
 - the language you are using
 - your name
 - the date you produced this standard (or revision)
-

Count type

Choices are logical and physical LOC.

- Logical LOC counts language elements.
- Physical LOC counts text lines.

For this counting standard, you are counting logical LOC.

Statement type

Use this section to define how you will count various types of statements.

Consider the following:

- How are you going to count procedure declarations and function prototypes?
 - How will you count compiler directives?
 - Will you count blank lines or comments? Why or why not?
-

Clarifications

A fully operational standard generally requires many notes and comments. Use the clarification section for this purpose.

Continued on next page

Pascal LOC Counting Standard Template

Definition Name: *Example Pascal LOC Std.*

Language: *Pascal*

Author: *W. S. Humphrey*

Date: *12/20/93*

Count Type	Type	Comments
Physical/Logical	<i>Logical</i>	
Statement Type	Included	Comments
Executable	<i>yes</i>	
Nonexecutable:		
Declarations	<i>yes</i>	
Compiler Directives	<i>yes</i>	
Comments	<i>no</i>	
Blank lines	<i>no</i>	
Other elements		
Clarifications		Examples/Cases
<i>Nulls</i>	<i>yes</i>	<i>continues, no-ops, ...</i>
<i>Empty statements</i>	<i>yes</i>	<i>“;;”, lone ;’s, etc.</i>
<i>Generic instantiators</i>		
<i>Begin...end</i>	<i>note 1</i>	<i>when executable</i>
<i>Begin...end</i>	<i>note 1</i>	<i>when not executable</i>
<i>Test conditions</i>	<i>yes</i>	
<i>Expression evaluation</i>	<i>yes</i>	<i>when used as sub program arguments</i>
<i>End symbols</i>	<i>notes 1,2</i>	<i>when terminating executable statements</i>
<i>End symbols</i>	<i>notes 1,2</i>	<i>when terminating declarations or bodies</i>
<i>Then, else, otherwise</i>	<i>note 1</i>	
<i>Elseif</i>	<i>note 1</i>	
<i>Keywords</i>	<i>notes 1,2</i>	<i>procedure division, interface, implementation</i>
<i>Labels</i>	<i>yes</i>	<i>branch destinations when on separate lines</i>
Note 1		<i>unless followed by ; or. or included in {}, count the following keywords once: BEGIN, CASE, DO, ELSE, END, IF, RECORD, REPEAT, THEN, UNTIL</i>
Note 2		<i>count every ; and . that is not within a {} or ()</i>
Note 3		<i>count each , between USES and the next ; or between VAR and the next ;</i>

Example 2: C++ LOC counting standard

**How many
LOC?**

Using the C++ LOC counting standard on page 7, how many LOC are in the following program fragment?

```
#include <stdio.h>

void main (void)
{
    int i,j;

    for (i = 0; i < 10; i++)
        for (j = 0; j < 10; j++)
            cout << i << ", " << j << endl;
}
```

Continued on next page

Example C++ Size Counting Standard

Definition Name: *Example C++ LOC std.*

Language: *C++*

Author: *W.S. Humphrey*

Date: *12/20/93*

Count Type	Type	Comments
Physical/Logical	<i>Logical</i>	
Statement Type	Included	Comments
Executable	<i>Yes</i>	
Nonexecutable:		
Declarations	<i>Yes, Notes 3, 4</i>	
Compiler Directives	<i>Yes, Note 4</i>	
Comments	<i>No</i>	
Blank lines	<i>No</i>	
Other elements		
Clarifications		Examples/Cases
<i>Empty statements</i>	<i>yes</i>	<i>";;", lone ;'s, etc.</i>
<i>Begin...end</i>	<i>note 1</i>	
<i>Expression evaluation</i>	<i>yes</i>	<i>when used as sub program arguments</i>
<i>End symbols</i>	<i>notes 1,2</i>	<i>for terminating executable statements, declarations, bodies</i>
<i>Then, else, otherwise</i>	<i>note 1</i>	
<i>Elseif</i>	<i>yes</i>	
<i>Keywords</i>	<i>yes</i>	<i>procedure division, interface, implementation</i>
<i>Labels</i>	<i>yes</i>	<i>branch destinations when on separate lines</i>
<i>Note 1</i>		<i>Count once every occurrence of the following key words: CASE, DO, ELSE, ENUM, FOR, IF, PRIVATE, PUBLIC, STRUCT, SWITCH, UNION, WHILE</i>
<i>Note 2</i>		<i>count once every occurrence of the following: ; , {} or {};</i>
<i>Note 3</i>		<i>count each variable or parameter declaration</i>
<i>Note 4</i>		<i>count once each #define, #ifdef, #include, etc. statement</i>

Example 3: another size counting standard

Overview

For this example standard, the class will select from among the following product categories.

- documents
- interface screens and forms
- database program elements
- maintenance fixes
- any other requested category

We'll then walk through developing the selected size counting standard.

Completing the standard

We'll then walk through the completion of the standard as a class exercise.

Continued on next page

Size Counting Standard Template

Definition Name: Programa 4a regla de simpson

Language: Java

Author: Marely Jacome Hernandez

Date: 27/11/25

Count Type	Type	Comments
Physical/Logical	Logical	Se contarán líneas lógicas, es decir, instrucciones reales del programa. No se cuentan líneas vacías ni comentarios. Cada instrucción debe ir en su propia línea para poder medir el tamaño del programa de forma consistente
Statement Type	Included	Comments
Executable	yes	Se cuentan: asignaciones, llamadas a métodos, condiciones if, bucles for/while, retornos, cálculos matemáticos, creación de objetos, iteraciones de Simpson, aplicación de Gamma, etc.
Nonexecutable:	yes	Declaraciones de clases, definiciones de métodos, firma del constructor, encabezados, y estructura del programa, porque forman parte de la lógica general del módulo PSP.
Declarations	yes	Declaración de variables como <code>double dblW;</code> , arreglos como <code>double[] dblFx;</code> , y objetos como <code>new GammaFunction();</code> cuentan como LOC lógicas individuales.
Compiler Directives	no	Java no utiliza directivas de preprocesador como <code>#include</code> o <code>#define</code> , por lo que no aplican en este programa.
Comments	no	No se cuentan como LOC. PSP exige que comentarios y documentación se excluyan para no afectar la métrica.
Blank lines	no	No se cuentan porque no representan lógica del programa.
Other elements	yes	Llaves solas <code>{ }</code> cuentan porque representan estructura lógica
Clarifications		Examples/Cases
		Una línea lógica = una instrucción completa <code>dblW = dblX / intNumSeg;</code>
		No se permiten múltiples instrucciones en una misma línea <code>i++; j++;</code>
		Declaraciones múltiples no permitidas: <code>int a, b, c;</code>
		Comentarios no cuentan como LOC: <code>// comentario</code> <code>/* comentario */</code>
		Las líneas lógicas de cálculo matemático cuentan <code>pow = Math.pow(base, dblExponent);</code> → 1 LOC <code>gammaNum = gammaFunction.computeIntGamma(n);</code> → 1 LOC <code>currentValue += dblFinalTerms[i];</code> → 1 LOC

Note		
Note		
Note		
Note		

Evaluation criteria and suggestions

**Evaluation
criteria**

Your standard must be

- complete
- legible

Suggestions

Keep your standards simple and short.

Do not hesitate to copy or build on the PSP materials.

Size Counting Standard Template

Definition Name: _____ Language: _____
 Author: _____ Date: _____

Count Type	Type	Comments
Physical/Logical		
Statement Type	Included	Comments
Executable		
Nonexecutable:		
Declarations		
Compiler Directives		
Comments		
Blank lines		
Other elements		
Clarifications		Examples/Cases
Note		
Note		
Note		
Note		